

# on time complexity and methods

A. Poojitha  
192511230

→ Compare time complexity of all notation in  
bubble, insertion, selection, merge, quick, heap

| → Name         | Best       | Average    | Worst      |
|----------------|------------|------------|------------|
| Bubble         | $n^2$      | $n^2$      | $n^2$      |
| Insertion      | $n^2$      | $n^2$      | $n^2$      |
| Selection Sort | $n^2$      | $n^2$      | $n^2$      |
| Heap Sort      | $n \log n$ | $n^3/2$    | $n^3/2$    |
| Heap           | $n$        | $n \log n$ | $n \log n$ |
| Quick          | $n \log n$ | $n \log n$ | $n^2$      |
| Merge          | $n \log n$ | $n \log n$ | $n \log n$ |

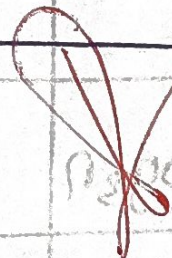
AT-1  
C04

101  
20

| Feature          | hashing                                              | binary search                                                    |
|------------------|------------------------------------------------------|------------------------------------------------------------------|
| Basic idea       | uses a hash function to map a key to an index $O(1)$ | $O(\log n)$<br>Repeatedly divides a sorted array into two halves |
| Data requirement | Does not require sorted data                         | Data must be sorted                                              |
| Best case        | $O(1)$                                               | $O(1)$                                                           |
| Average case     | $O(1)$                                               | $O(\log n)$                                                      |
| Worst case       | $O(n)$ due to collision                              | $O(\log n)$                                                      |



| Search method | Hash function + hash Table | Compare with middle element              |
|---------------|----------------------------|------------------------------------------|
| Insertion     | Average $O(1)$             | $O(n)$ in an array if maintaining sorted |
| Deletion      | Average $O(1)$             | $O(n)$ in an array                       |
| Main issue    | Collision handling         | Requires sorted data                     |



(array)  
 repeated elements  
 in sorted array  
 into two halves  
 left half & right half

(array)  
 uses a hash  
 function to  
 map a key to  
 an index (0)

(array)  
 uses a hash  
 function to  
 map a key to  
 an index (0)